

OBJEKTORIENTERET ANALYSE, DESIGN OG IMPLEMENTATION

7: Threads, concurrency and synchronization

Jan H. Mikkelsen

Cyber og Computerteknologi

2. semester – KBH – F26

AGENDA

- Multitasking
 - Processes and threads
 - Creating threads in Java
 - Thread life-cycles and methods
- Concurrency
 - Problems and solutions
 - Message sequence diagrams
 - Thread synchronization
- Examples
- The important stuff
- Assignments

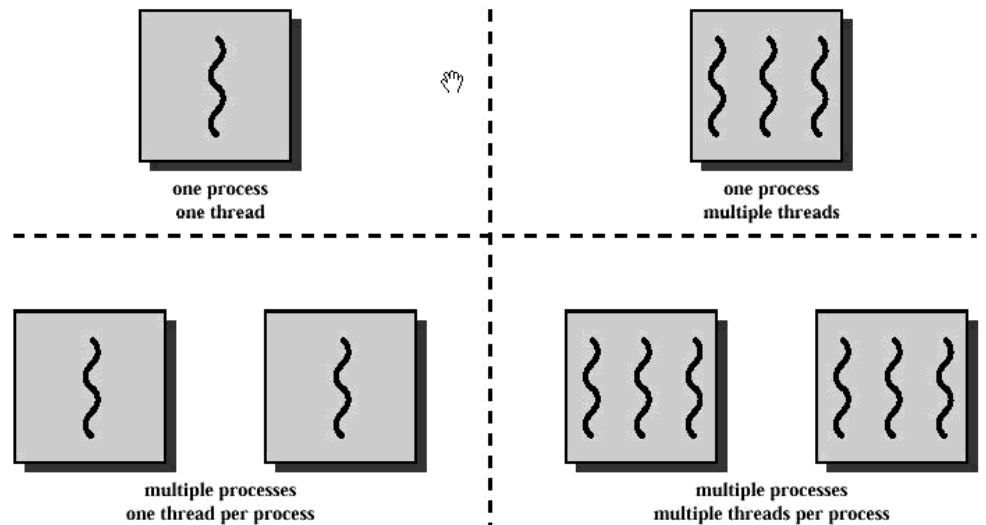


- To be able to understand and use threads
- To be able to understand and design for concurrency
- To be able to understand synchronization issues in relation to concurrency

PROCESSES VERSUS THREADS

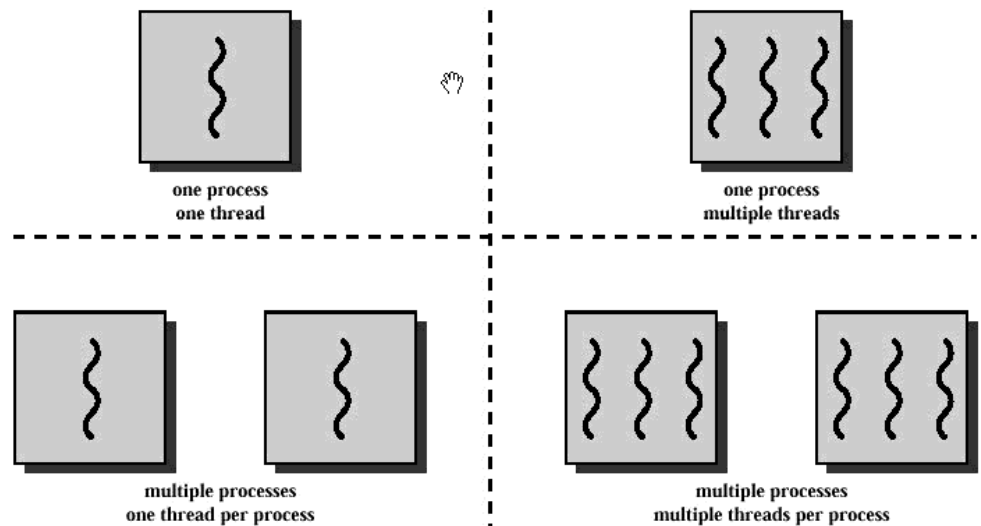
2

- A 'Process' is a program in execution
- A 'Thread' is a dispatchable unit of work
 - It sometimes is considered a light-weight processes-within-a-process
- As such we can say, that a process may contain any number of threads
- Memory is the critical distinction
 - Processes are isolated from each other by having separate memory spaces
 - Threads within the same process share memory, which is both a strength and a great danger



PROCESSES VERSUS THREADS

- Every Java program already has threads, and even a simple “Hello World” runs on the main thread
- In addition, the JVM runs background threads for garbage collection, for instance
- Java threads map to native OS threads in modern JVMs, meaning the OS scheduler handles them directly
- Java's "write once run anywhere" promise extends to threading and the same code runs on any OS despite scheduling differences



MULTITHREAD-BASED MULTITASKING

- To recapture, let us assume two applications running on a computer results in the creation of two processes
- Both processes live completely separate lives
- Processes operate on separate address space, where each process has its own data area, code area, and dynamic heap and stack memory areas
- Process-based multitasking is when two or more processes run simultaneously and implements multi-tasking
- This is controlled by the OS in question

Separate address spaces provide protection. If a browser tab crashes, it cannot corrupt your word processor's memory. The OS enforces this boundary at the hardware level. This is a deliberate design decision, not a limitation.

MULTITHREAD-BASED MULTITASKING

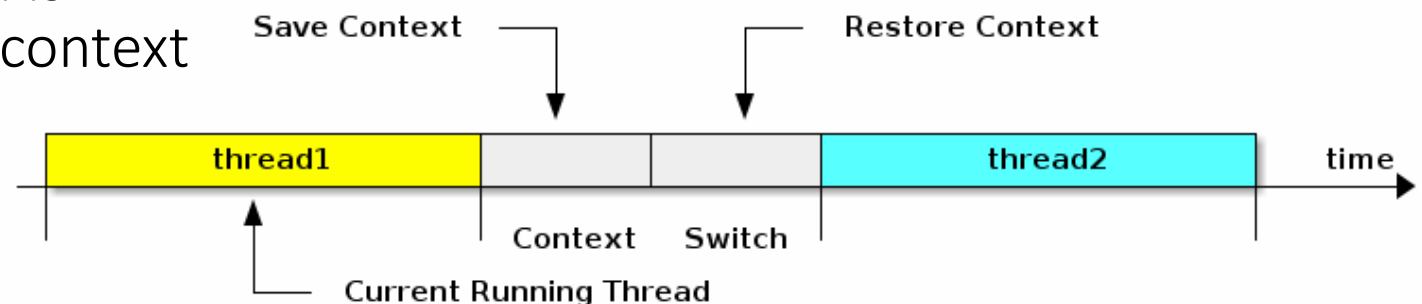
5

- In contrast to this we have multithread-based multitasking
- This is when two or more threads run simultaneously and implements multi-tasking
- Threads belonging to the same process share the same address space and share the same data, same code and heap area. However, there are different stack areas for each thread
- This is implementable by programming
- This is supported by Java
 - Threads share the same address space
 - Context-switching between threads is ~inexpensive
 - Communication between threads is ~inexpensive

The separate stack per thread is important. Each thread has its own execution path, its own method calls, its own local variables, its own call history. If threads shared a stack, one thread calling a method would corrupt another thread's local variables. Stack separation is not optional - it is architecturally necessary for threads to work at all

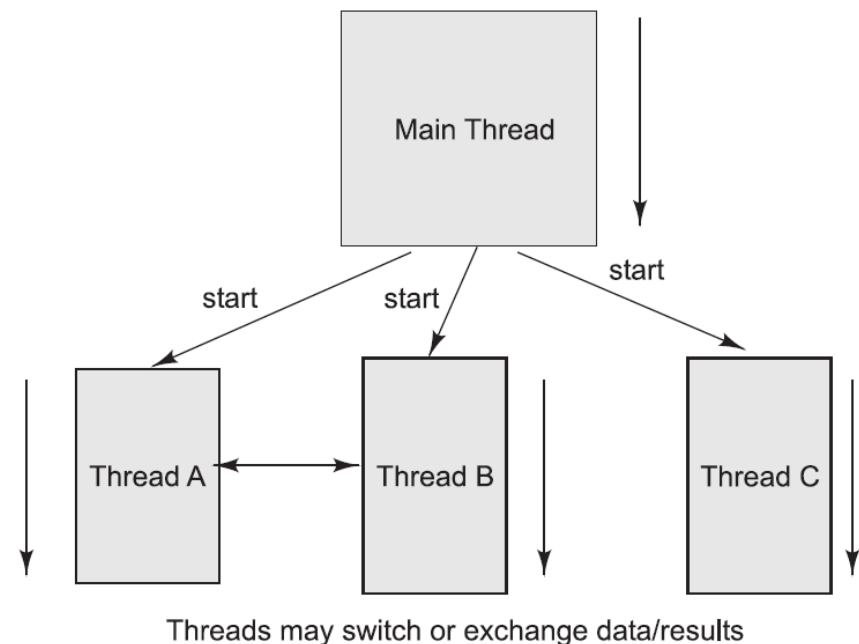
CONTEXT-SWITCHING

- When the OS or JVM scheduler decides to pause one thread and run another, it needs to save the complete state of the paused thread so it can resume exactly where it left off later
- That saved state is called the context, and the act of saving one and restoring another is the context switch
- Cooperative versus preemptive multitasking
- Timeslices regulated by HW timers
- Thread information is saved onto stacks (context structures)



THREADS IN JAVA

- A single process/application might contain multiple threads
- All threads within a process share the same state and memory space, and can communicate with each other directly – because they share the same variables
- Threads are absorbed/destroyed automatically* when they are no longer needed
- The scope of a thread is within a specific code module



*) User threads keep running in the JVM until they are all finished once their `run()` method completes. Daemon threads — background threads (like garbage collection) that the JVM destroys automatically when no user threads remain. If you forget to terminate a user thread, your application will not shut down cleanly

CREATING THREADS IN JAVA

- A thread's work is defined by a unit of code, specifically there are two options:
 - Create a class that extends the Thread class
 - Simpler and more readable but is limited by single inheritance. If a class extends Thread it cannot extend anything else
 - Create a class that implements the standard Runnable interface
 - The class remains free to extend another class and it achieves better separation of task (what to do) and the thread (the mechanism of doing it)
- The run() method should be overridden and should contain the code that will be executed by the new thread
- This method must be public with a void return type and should not take any arguments

```
MyThread t = new MyThread();  
t.run(); // WRONG - runs on current thread, no new thread created  
t.start(); // RIGHT - creates new thread, which then calls run()
```

<https://docs.oracle.com/javase/tutorial/essential/concurrency/runthread.html>

EXTENDING THE THREAD CLASS

9

```
class MyThread extends Thread {  
    public void run() {  
        // thread body of execution  
    }  
}
```

Create a class by
extending the Thread class
and override the run()
method



```
MyThread thr1 = new MyThread();  
thr1.start();
```

Create thread
object



Start execution



<https://docs.oracle.com/javase/7/docs/api/java/lang/Thread.html>

IMPLEMENTING THE RUNNABLE INTERFACE

```
class MyThread implements Runnable {  
    ...  
    public void run() {  
        // thread body of execution  
    }  
}
```

Create a class by implementing the Runnable interface and override the run() method

```
MyThread myObject = new MyThread();
```

Our task object

```
Thread thr1 = new Thread(myObject);
```

Wrap it in a Thread

```
thr1.start();
```

Start execution

<https://docs.oracle.com/javase/7/docs/api/java/lang/class-use/Runnable.html>

IMPLEMENTING THE RUNNABLE INTERFACE

11

```
class MyThread implements Runnable {  
    ...  
    public void run() {  
        // thread body of execution  
    }  
}
```

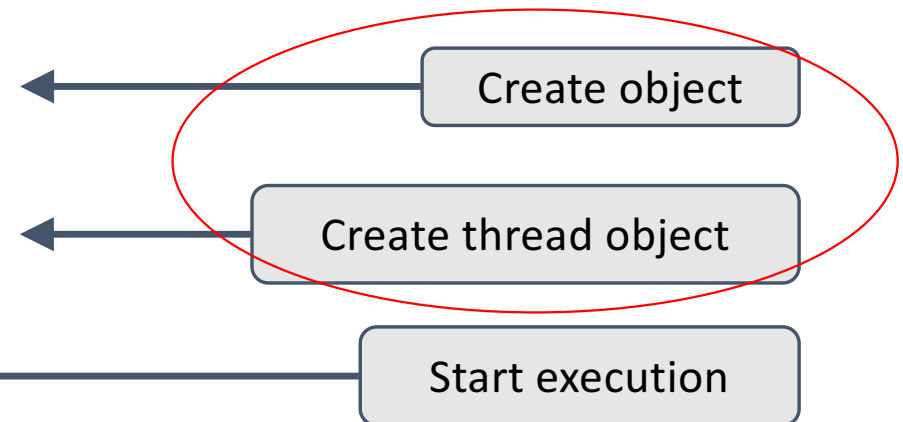
Three steps are needed here, and not two as before

Separation of the “what” and the “mechanism that does it”

```
MyThread myObject = new MyThread();
```

```
Thread thr1 = new Thread(myObject);
```

```
thr1.start();
```



<https://docs.oracle.com/javase/7/docs/api/java/lang/class-use/Runnable.html>

DIFFERENCES BETWEEN THE TWO METHODS

- Method 1 (extending the Thread class)
- The derived class cannot extend any other base class
- Used when the program requires a full control over the thread life cycle

- Method 2 (implementing the Runnable interface)
- Does not give any control over the thread itself to the developers
- Good choice if the program needs flexibility for extending other base classes
- Preferred approach

LAMBDA EXPRESSIONS

- A note on “modern Java” and the implementation of Runnable
- Java – at some point – introduced the lambda expressions, which makes Runnable even cleaner

```
MThread thr1 = new Thread(() -> {  
    // thread body here  
});  
thr1.start();
```

versus

```
MyThread myObject = new MyThread(); // your task object  
Thread thr1 = new Thread(myObject); // wrap it in a Thread  
thr1.start(); // start the thread
```

LAMBDA EXPRESSIONS

14

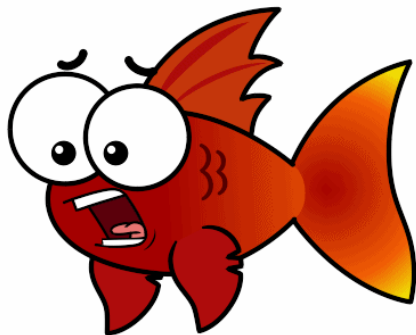
- A lambda expression is simply a method without a name .. that is it
- Both block implement exactly the same functionality, with the top one using a load of pompous and ceremonial fanfare
- The bottom one gives the same result but with far less noise
- They work with functional interfaces only. That is interfaces that have exactly one abstract method

```
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Hello from thread");  
    }  
}  
  
Thread t = new Thread(new MyTask());  
t.start();
```

```
Thread t = new Thread(() -> System.out.println("Hello from thread"));  
t.start();
```

BREAK – PAUSE(FISK)

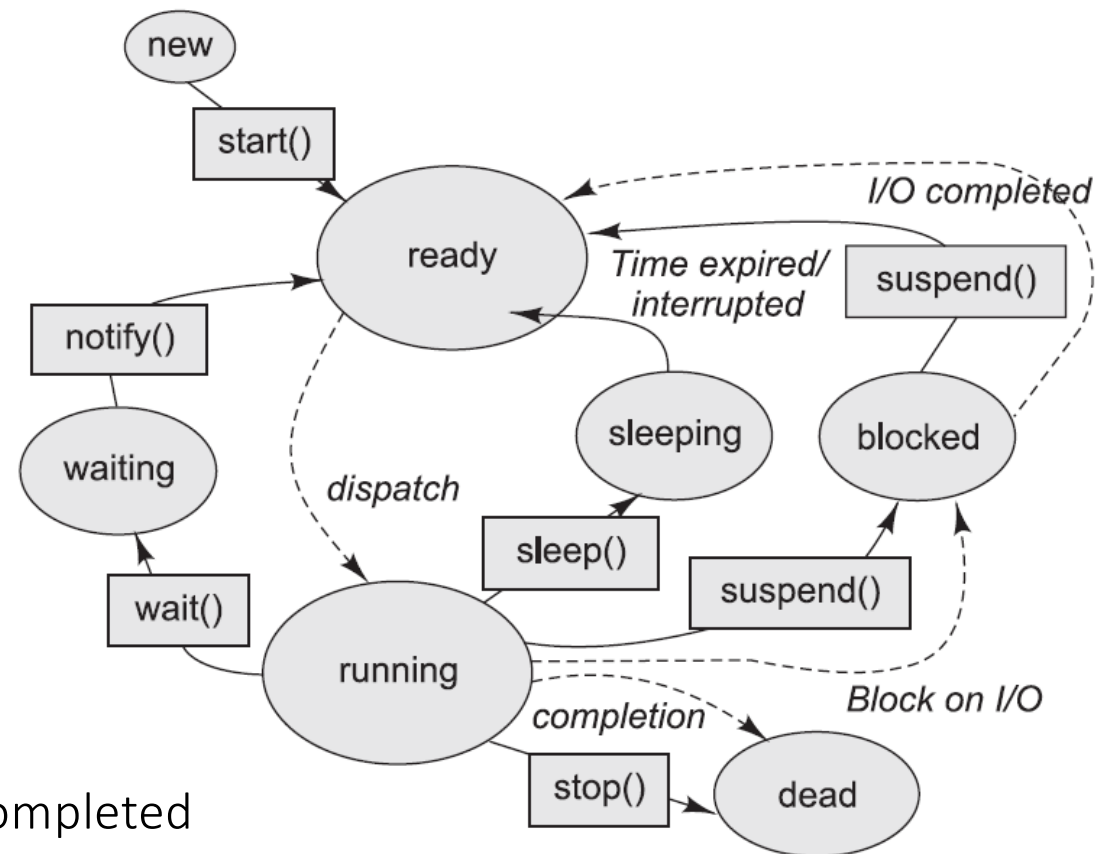
15



THREAD LIFE-CYCLE

16

- The life cycle of the thread in java is controlled by the Java Virtual Machine (JVM)
- A thread moves from one state to another depending on the operation performed by or on it
- Four states in total
 - New – created, not called
 - Runnable – after start()
 - Blocked/waiting
 - Terminated/dead – run() has completed



USEFUL THREAD METHODS

- `public static Thread currentThread()`
 - returns the thread that is currently executing
- `public void interrupt()`
 - Interrupts the called thread, so another can get a change to be executed
- `public void suspend()`
 - Suspends the current thread – can be resumed later.
- `public void resume()`
 - Resumes a thread from suspended status
- `public void setPriority(int priority)`
 - Sets a specified priority to the thread: 1-10
- `public int getPriority()`
 - Returns the priority of the thread: 1-10
- `public void sleep(long millis)` throws `InterruptedException`
 - Causes a running thread to sleep for a specified amount of time in milliseconds

Deprecated in modern Java. If a thread is suspended while holding a lock, that lock is never released and we have a deadlock on our hands. In modern Java they are replaced by `wait()` and `notify()`

<https://docs.oracle.com/javase/7/docs/api/java/lang/Thread.html>

OTHER USEFUL THREAD METHODS

- Another set of methods are used mainly for solving concurrent access issues related to
 - Thread Synchronization
 - Implementation of semaphores
- `public final void wait()`
- `public final void wait(long millis)`
- `public void notify()`
- `public void notifyAll()`
- `public final void join()`
- `public final void join(long millis)`

We will return to these
a bit later

`wait()` — thread stops and releases its lock, waiting until notified.
`notify()` — wakes up one waiting thread. `notifyAll()` wakes all waiting threads
`join()` — makes the calling thread wait until another thread finishes

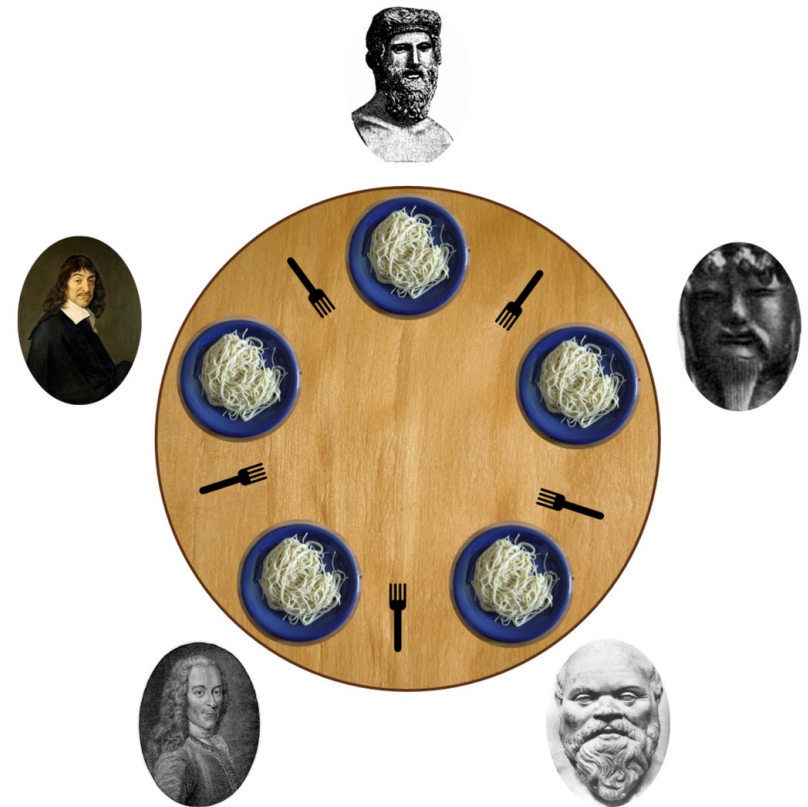
<https://docs.oracle.com/javase/7/docs/api/java/lang/Thread.html>

AGENDA

- Multitasking
 - Processes and threads
 - Creating threads in Java
 - Thread life-cycles and methods
- Concurrency
 - Problems and solutions
 - Message sequence diagrams
 - Thread synchronization
- Examples
- The important stuff
- Assignments

THE DINING PHILOSOPHERS

- One of the most famous problems in computer science
- Each philosopher can only
 - a) eat, or
 - b) think
- Each philosopher needs two forks to eat (do not question this), one for each hand
- How do we avoid them getting hungry?
- Concurrent resource contention
- Forks are shared objects that we need to lock and release

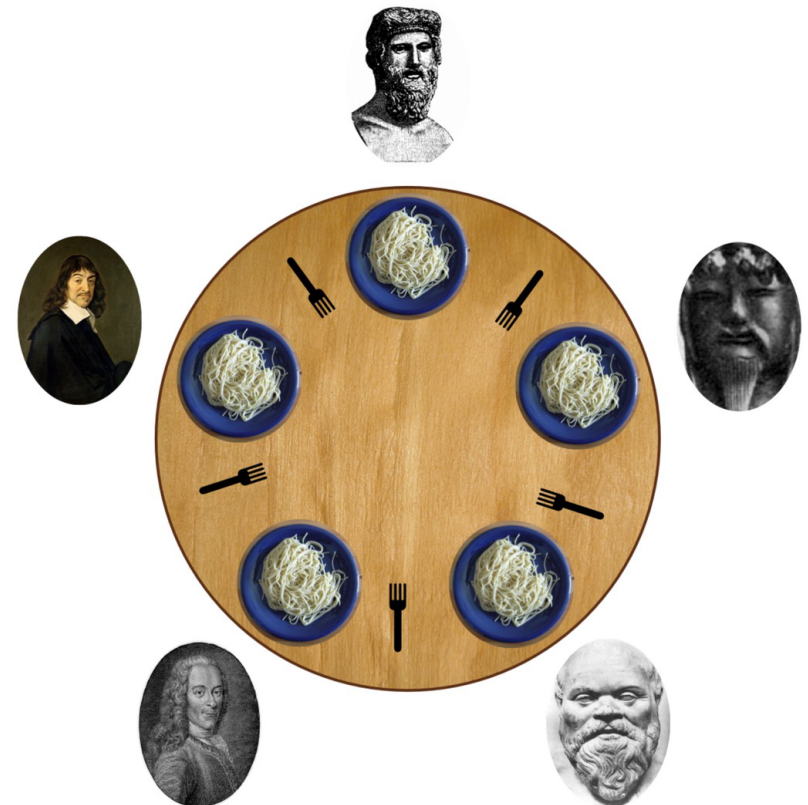


RACE CONDITIONS AND CRITICAL SECTIONS

- Race Conditions
 - Occurrence of a condition where two or more threads can access shared data and then try to change its value at the same time
 - Due to this, the values of variables may be unpredictable and vary depending on the timing of context switches of the processes
- How to solve it?
 - Use thread synchronization, which is a mechanism that ensures that two or more concurrent threads do not simultaneously execute some particular program segment known as critical section
- Critical sections
 - These are the parts of the program where the shared resource is accessed

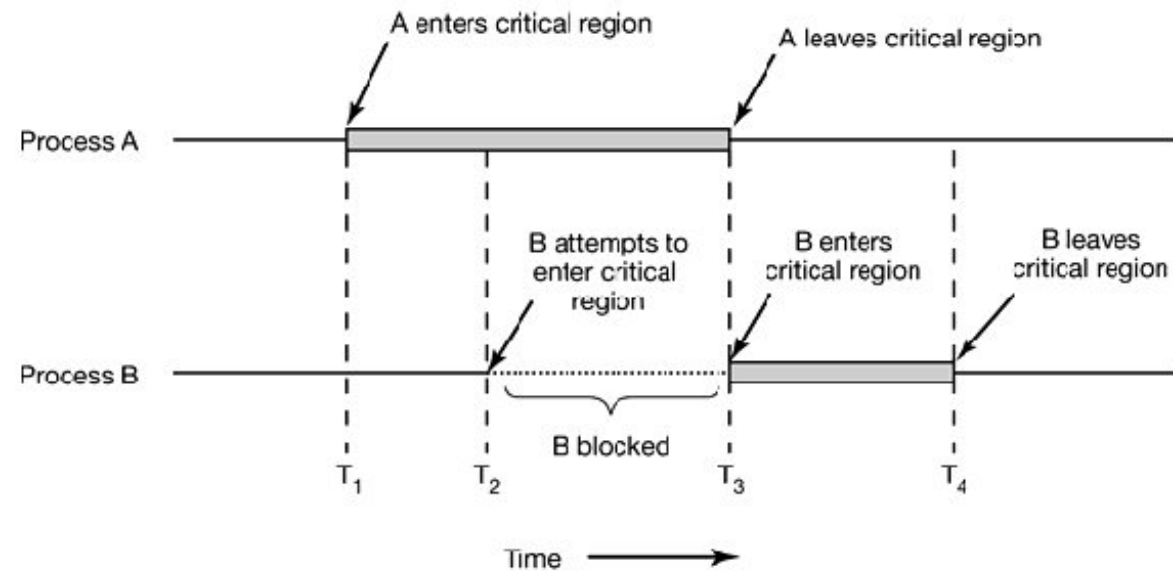
THE DINING PHILOSOPHERS

- Returning to the dining philosophers we can now express the problem in (more) technical terms
- Again .. the forks are shared resources
- Picking up a fork is entering a critical section
- The philosophers are threads in a race for the same resources
- Deadlock is what happens when the critical section solution is implemented incorrectly



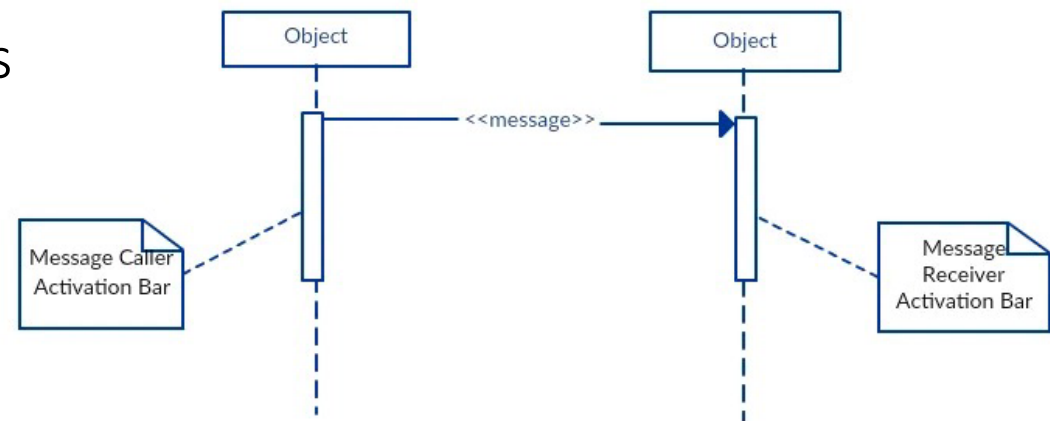
DESIRED PROCESS BEHAVIOUR

Race Conditions , Critical sections, & Mutual Exclusion



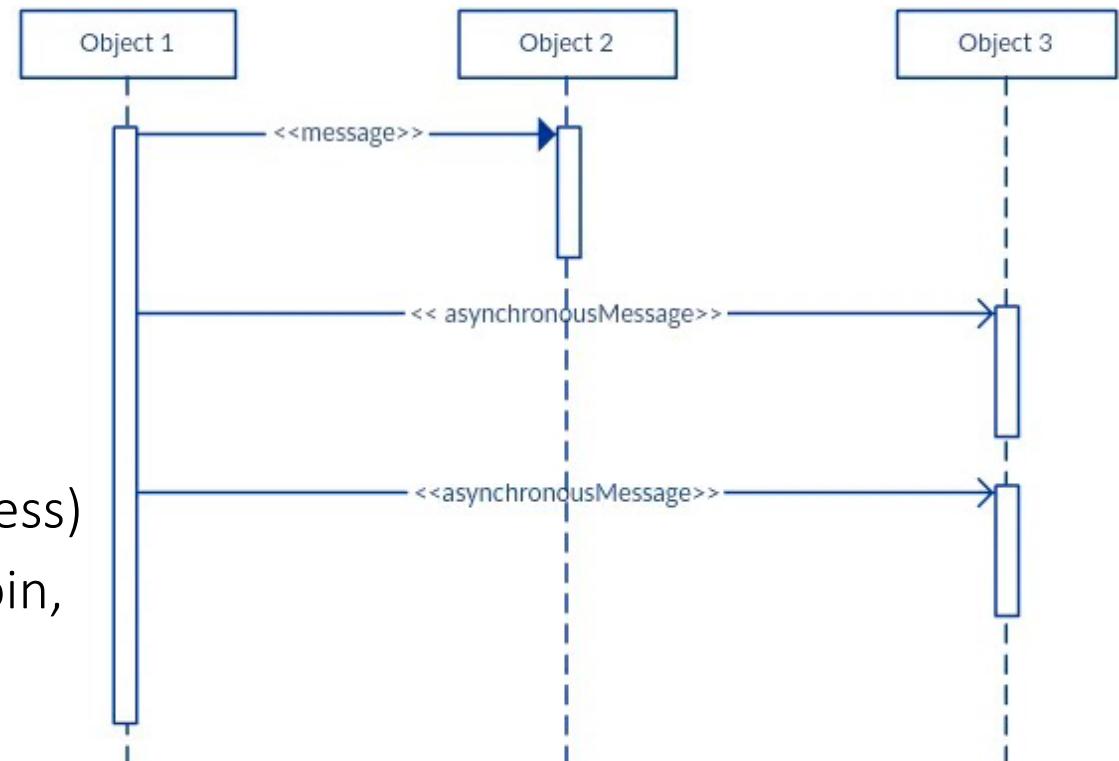
MESSAGE SEQUENCE DIAGRAMS

- Message sequence diagrams model behaviour over time
- A class diagram shows structure while a MSD shows dynamics
- Activation bars are here used to indicate that objects are active in some period of time
- Typically, a note on the activity is added
- Exchange of information between objects is shown by an arrow with text (indicating direction of information flow)
- Can also be a method call. Object A sending a message to object B, is essentially A spawning a thread to handle B's work independently



SYNCHRONOUS VERSUS ASYNCHRONOUS MESSAGES

- Synchronous messaging
 - Caller waits (blocked until reply)
 - Thick arrow-head
- Asynchronous messaging
 - Caller “fires and forgets”
 - Thin arrow-head
- Particularly for asynchronous messaging the diagram is only one example of a realization (simplified randomness)
- Code must coordinate; await, join, use callbacks, or design for eventual consistency



BANKING EXAMPLE

- The scenario is a simple bank account

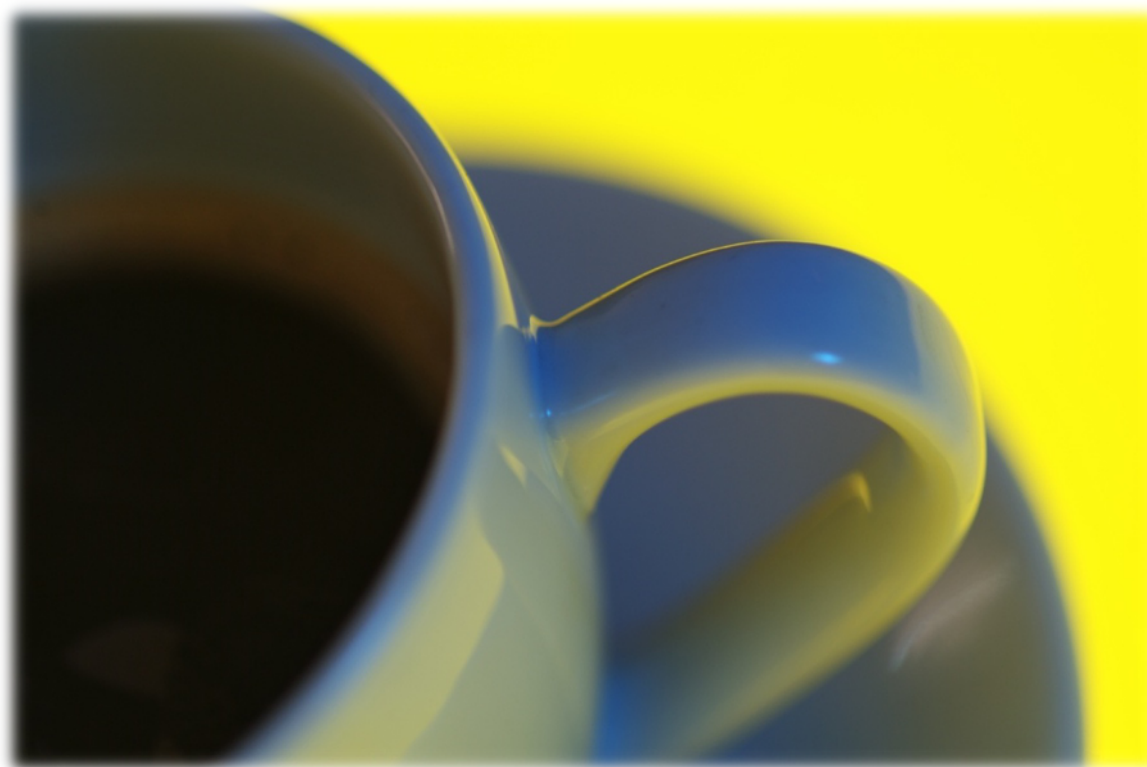
```
public class BankAccount {  
    private double balance = 1000.0;  
  
    public void deposit(double amount) {  
        balance = balance + amount;  
    }  
  
    public void withdraw(double amount) {  
        balance = balance - amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

BANKING EXAMPLE

- Here we have two threads that make identical deposits and withdrawals
- The starting balance was 1000 so after running this code we should ALWAYS expect a balance of 1000
- However, if we run the code we will see varying results; 800, 1000 (by chance), 1200 etc.

```
public class BankTest {  
    public static void main(String[] args) throws InterruptedException {  
        BankAccount account = new BankAccount();  
        // Thread 1 - deposits £100 a thousand times  
        Thread t1 = new Thread(() -> {  
            for (int i = 0; i < 1000; i++) {  
                account.deposit(100.0);  
            }  
        });  
        // Thread 2 - withdraws £100 a thousand times  
        Thread t2 = new Thread(() -> {  
            for (int i = 0; i < 1000; i++) {  
                account.withdraw(100.0);  
            }  
        });  
        t1.start();  
        t2.start();  
        t1.join(); // wait for both to finish  
        t2.join();  
    }  
}
```

"BIKER BREAK!"

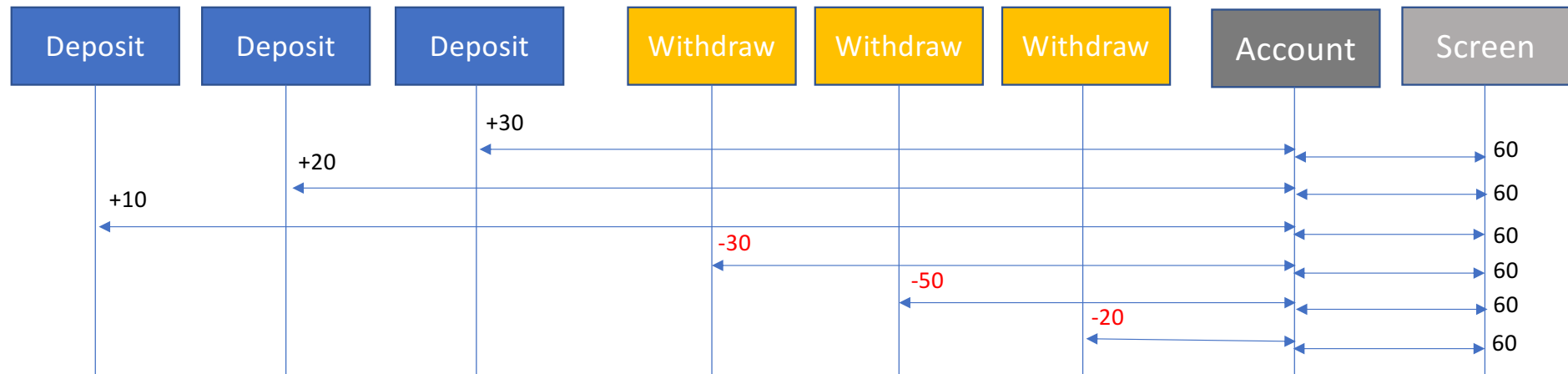


Jan H. Mikkelsen, OOADI7, CoCT, 2026

BANKING APPLICATION – WHAT JUST HAPPENED?

29

- Automated (fast) operation and not Aunty Oda doing it manually 😊

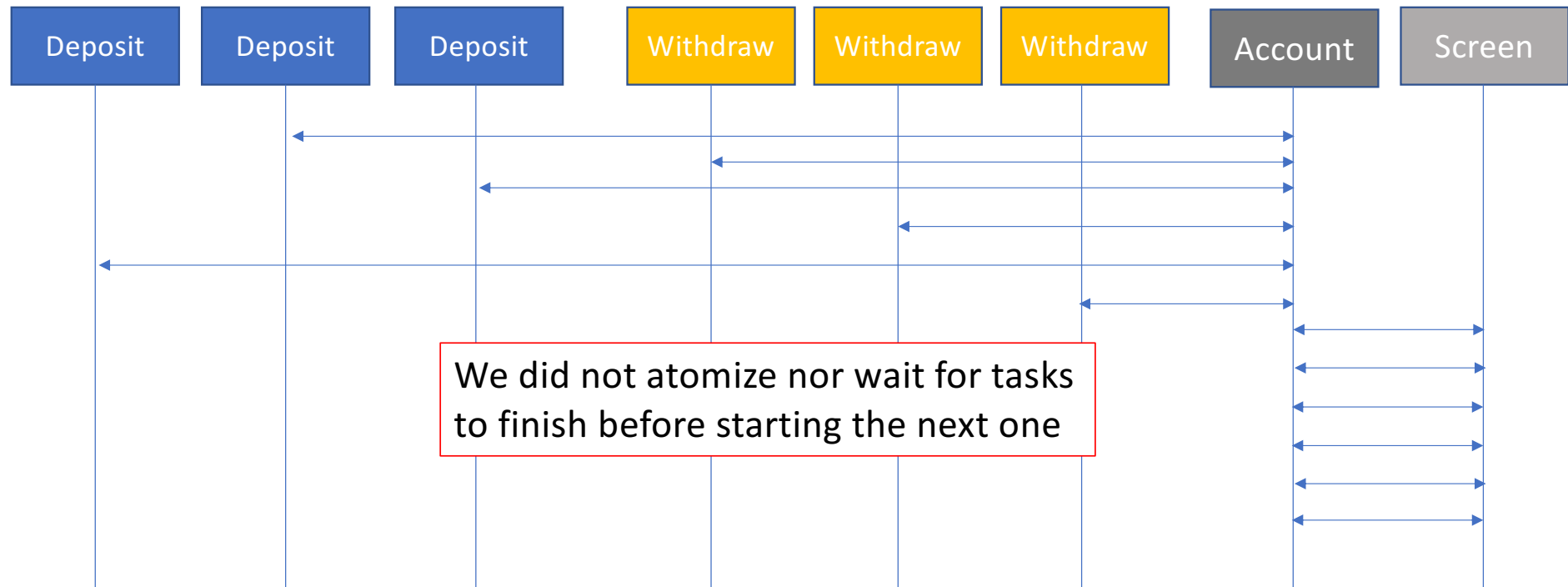


➤ Nice account, I always have 60 credits?!?!? Or do I...? What happens???

- Due to thread management, execution is not ordered sequentially
 - In this case, it appears that the output is written AFTER executing transactions but before account is updated

BANKING APPLICATION – PROBABLY THE HARSH REALITY

30



ATOMIZING

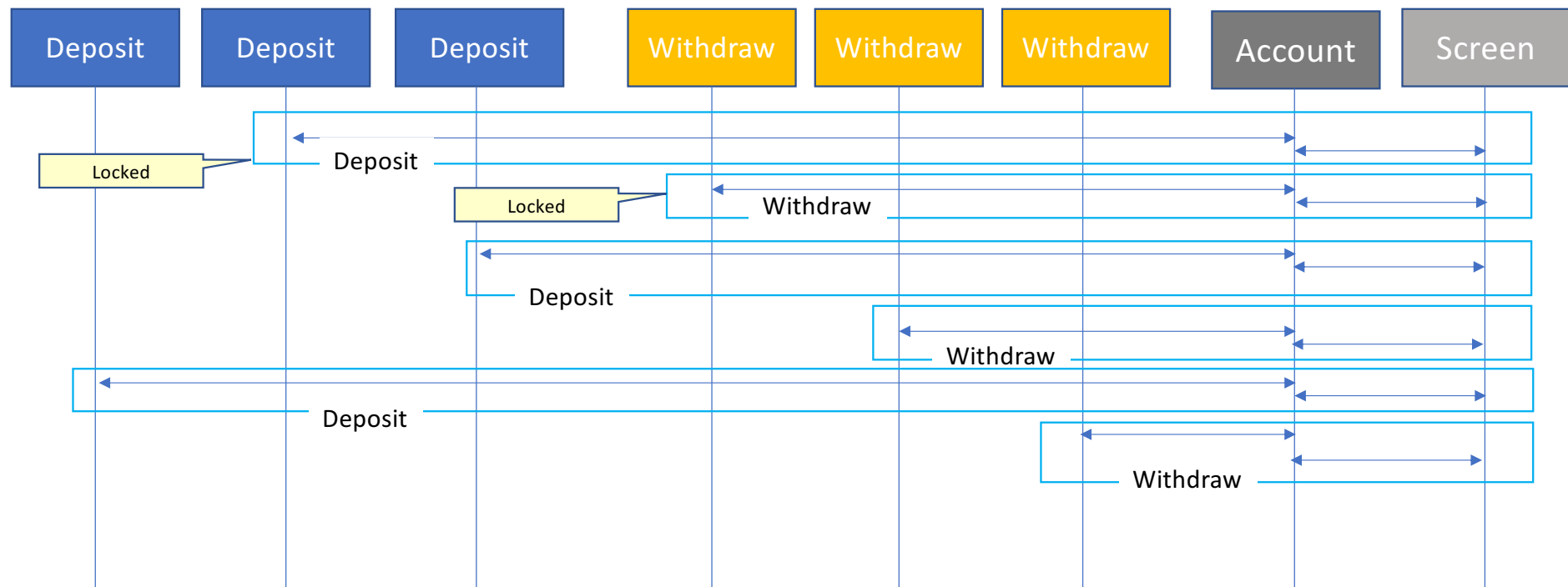
counter = counter + 1; // NOT atomic: it is 3 operations!

```
synchronized void increment() {  
    counter = counter + 1; // NOW atomic  
}
```

```
AtomicInteger counter = new AtomicInteger(0);  
counter.incrementAndGet(); // atomic, lock-free
```

BANKING APPLICATION - SYNCHRONIZED

32



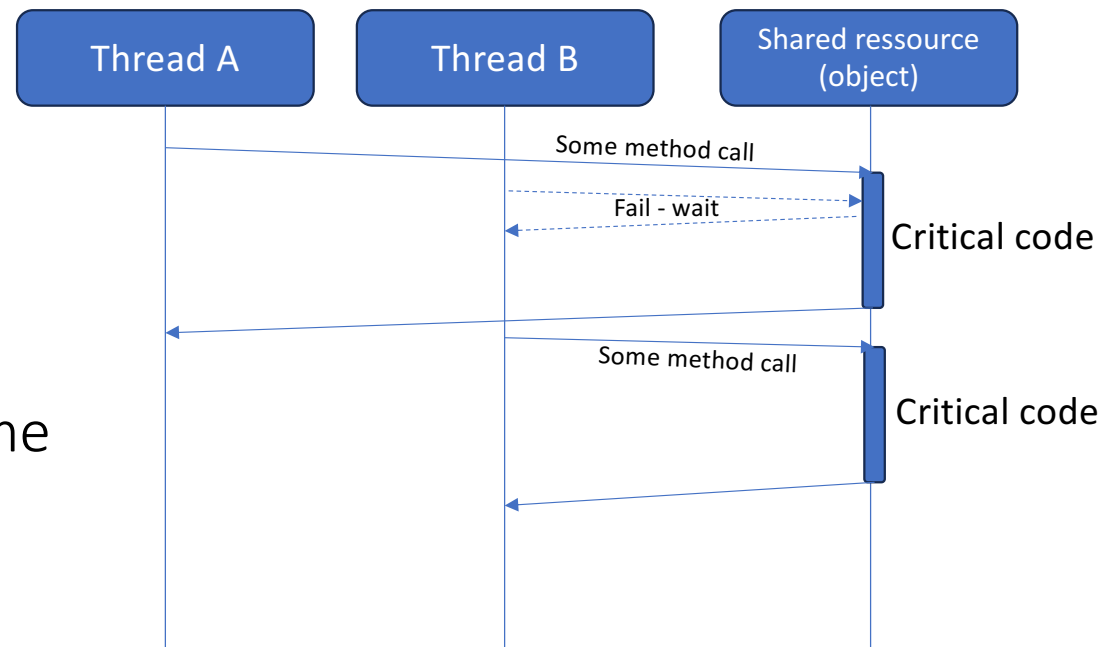
SYNCHRONIZED METHODS AND CODE BLOCK

33

- Unlike C, where we talk about mutexes and semaphores, then we work with synchronized methods in Java

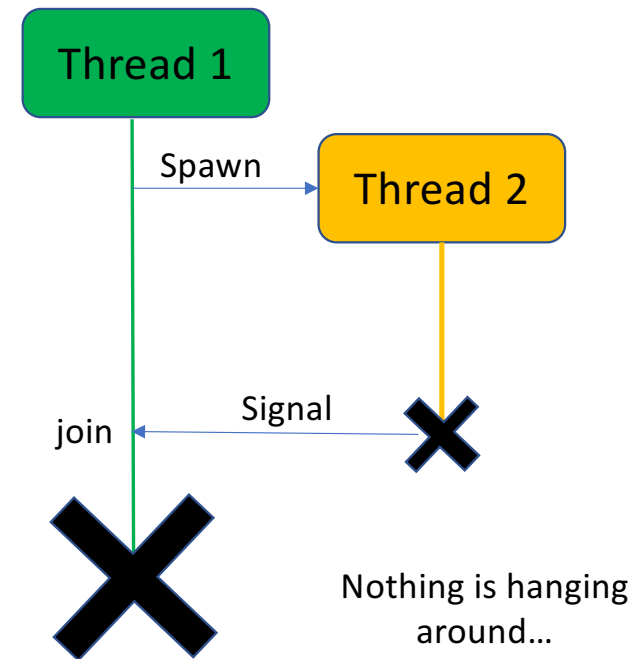
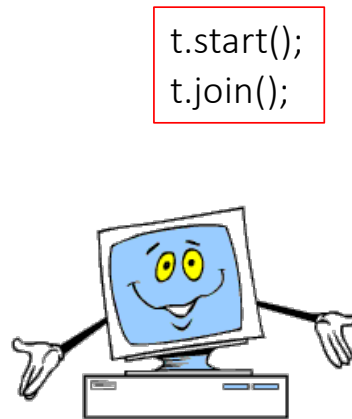
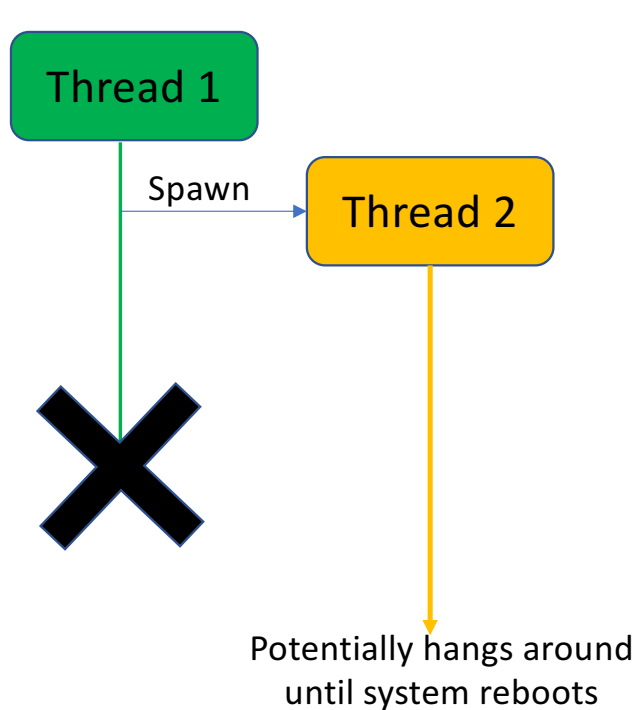
```
public synchronized output methodName(){  
    ... critical code  
}
```

- Critical code?
- Shared variables, objects, files, any resources with one-at-a-time access



JOIN – WHAT IS IT GOOD FOR?

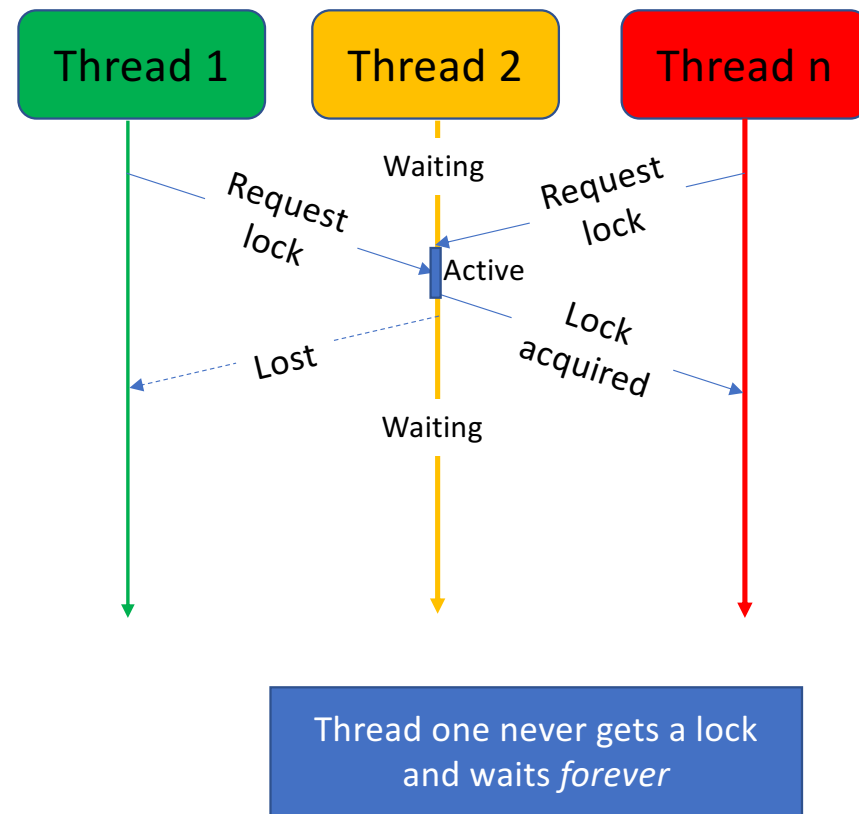
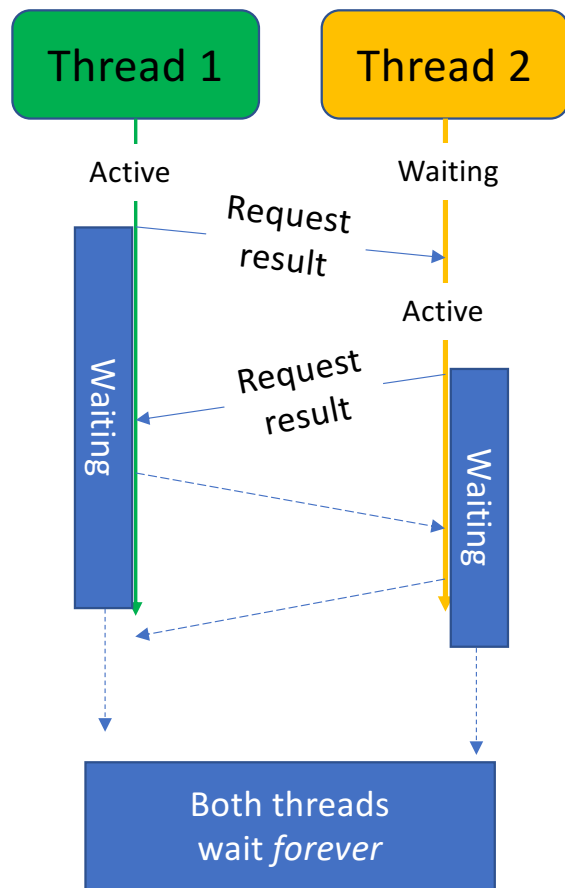
34



- Particular important when resources have been locked
- Can also be used to control order of thread execution...

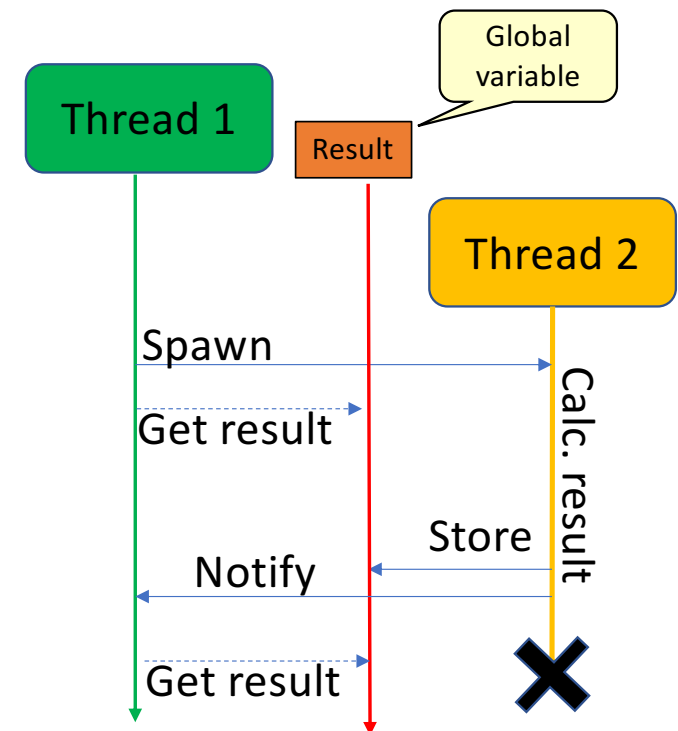
WAITING/NOTIFYING PITFALL EXAMPLES

35



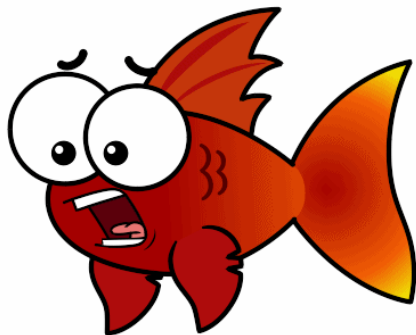
WAITING FOR OTHERS – WAIT()

- It is easy to make this mistake
- Assume results are created in similar fashion as with functions/method (sequentially)
- At best we get 0 or something not meaningful, worse we get something meaningful
- We therefore need to wait for results to be produced, obtained or ready
- Using Notify and Wait we can (elegantly) ensure synchronization of threads
- `join() != wait()`
 - `join()` = “I want *that* thread to finish.”
 - `wait()` = “I’m temporarily pausing **inside a synchronized block** and giving others a chance to update the shared condition.”



BREAK – PAUSE(FISK)

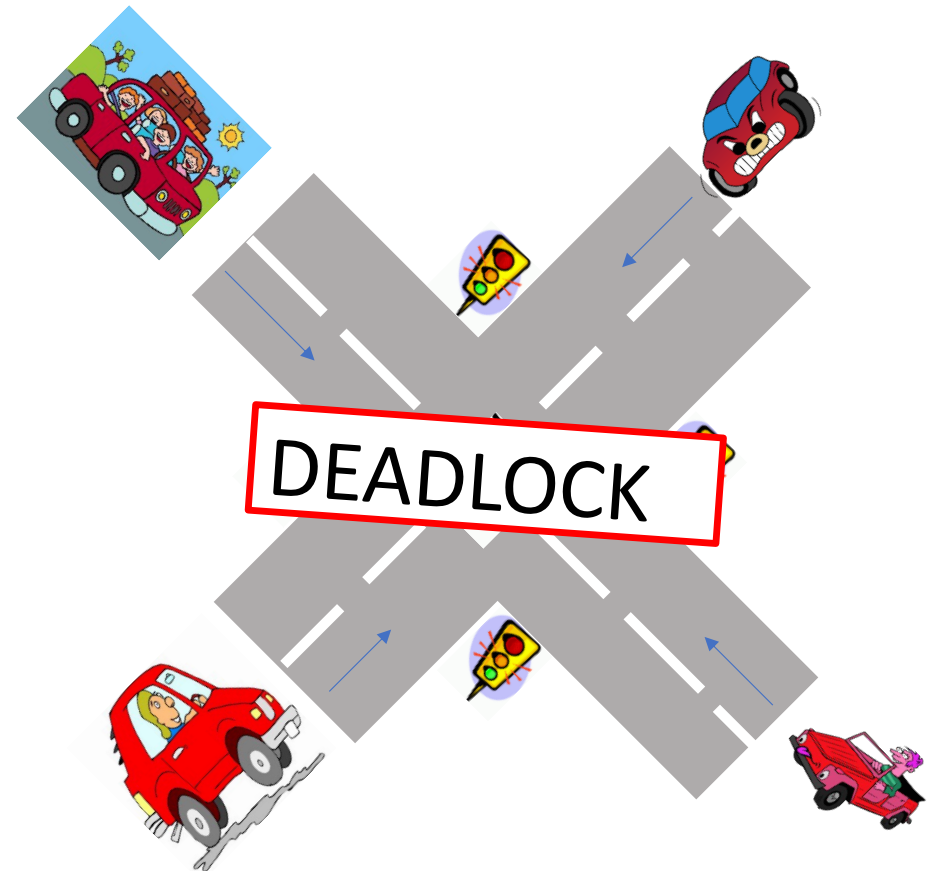
37



Jan H. Mikkelsen, OOADI7, CoCT, 2026

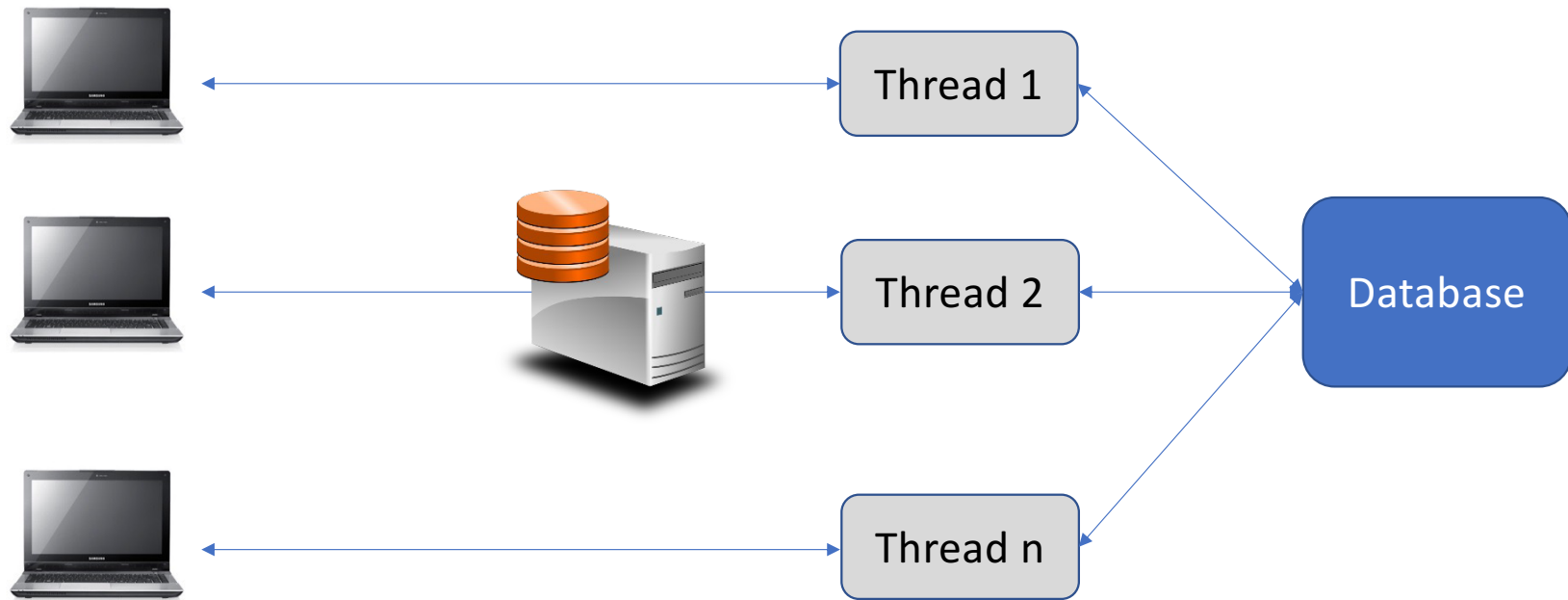
DEADLOCK IN A NUTSHELL

- Deadlocks can occur when two (or more) threads are blocking each other's progress
- Imagine you have two locks, L1 and L2, and one thread tries to acquire L1 followed by L2, whereas another thread tries to acquire L2 followed by L1
 - Thread 1: Acquire and hold L1.
 - Thread 2: Acquire and hold L2.
 - Thread 1: Try to acquire L2, but it's in use, so wait.
 - Thread 2: Try to acquire L1, but it's in use, so wait.
 - The threads are now deadlocked: no execution will proceed.



WE NEED TO SYNCHRONIZE THREADS

- We need to ensure atomic execution of code – using locks, semaphores and condition variables



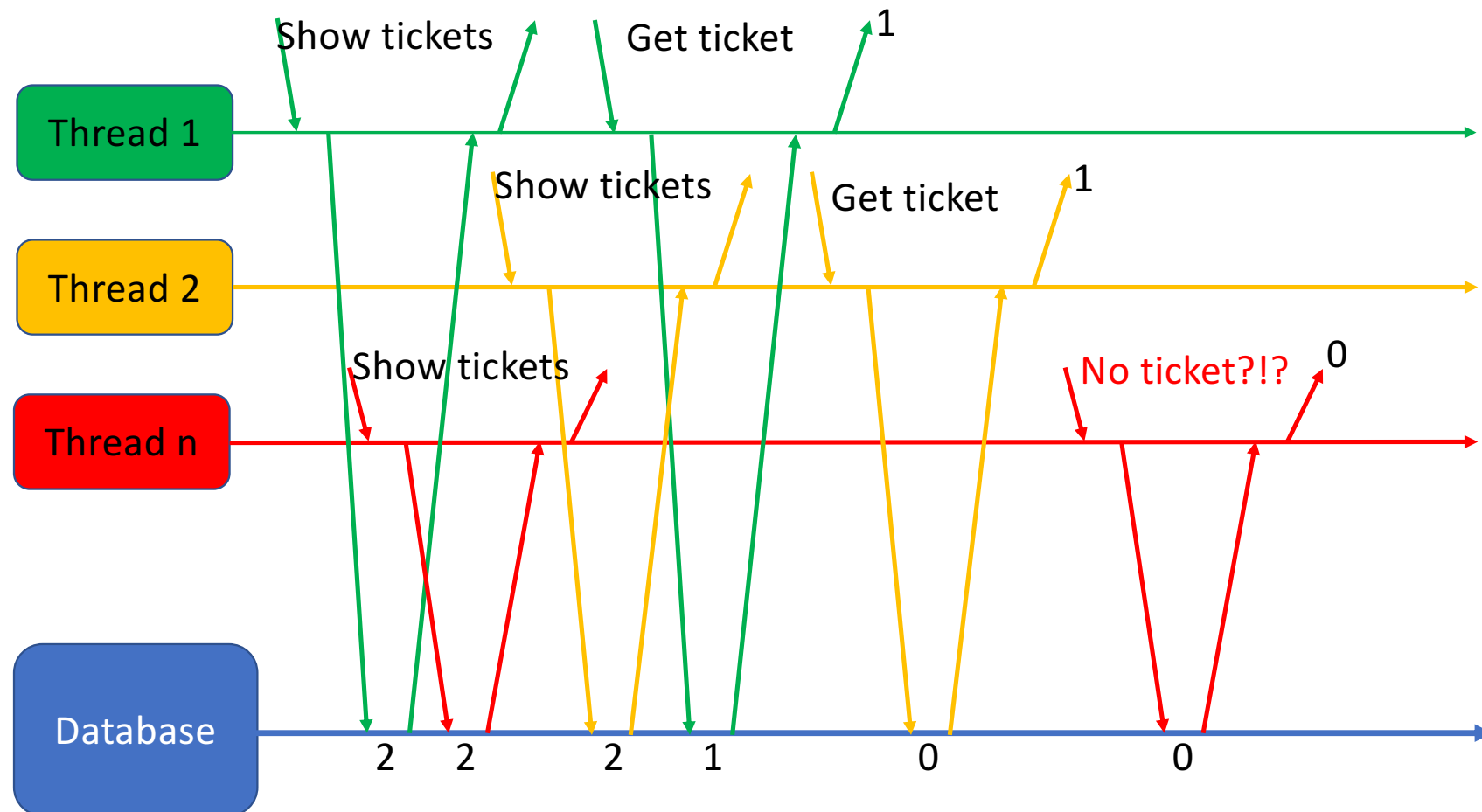
AGENDA

40

- Multitasking
 - Processes and threads
 - Creating threads in Java
 - Thread life-cycles and methods
- Concurrency
 - Problems and solutions
 - Message sequence diagrams
 - Thread synchronization
- Examples
- The important stuff
- Assignments

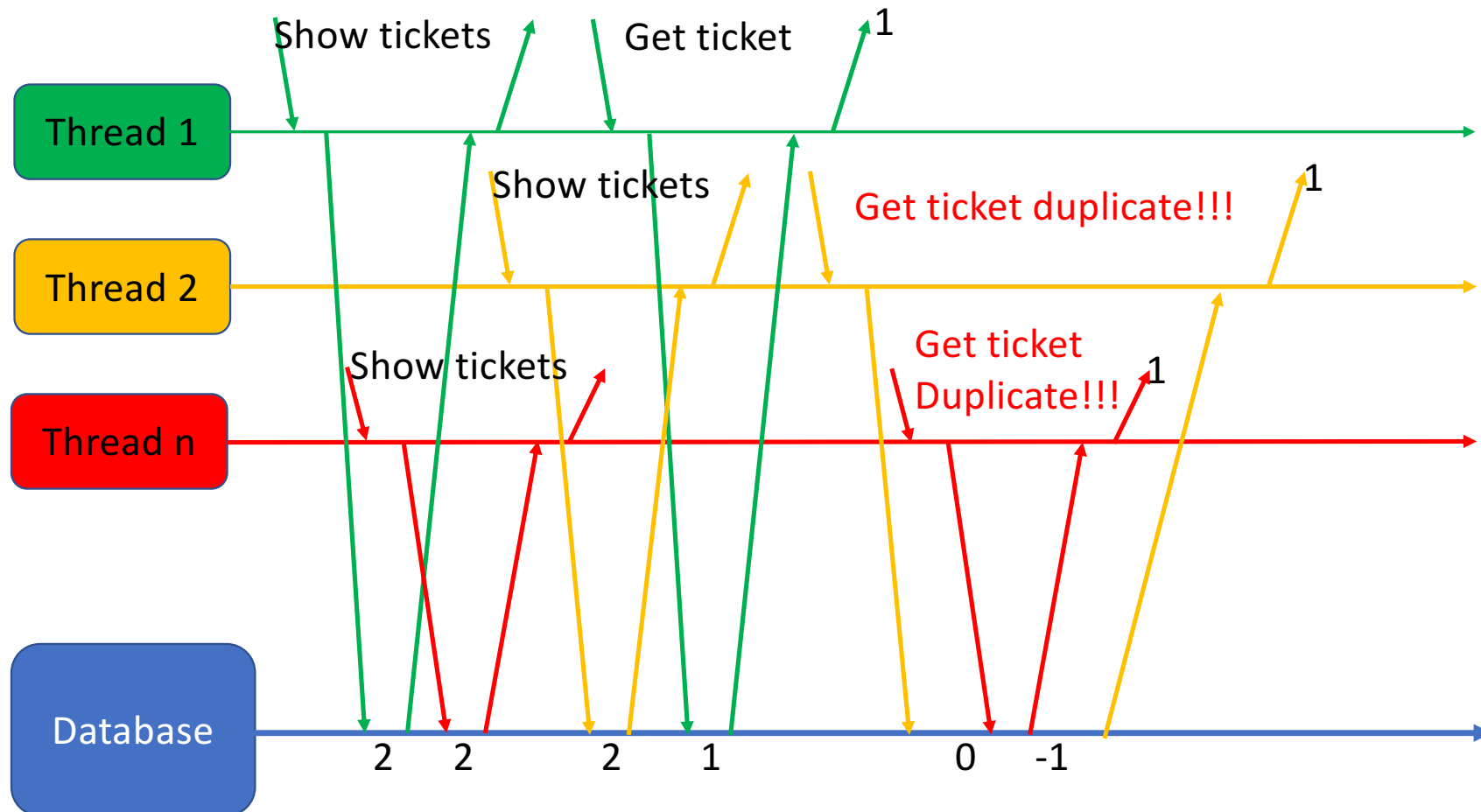
THE PROBLEM IS RELATED TO TIME (A)

41



EVEN WORSE – BUYING NONEXISTING STUFF (B)

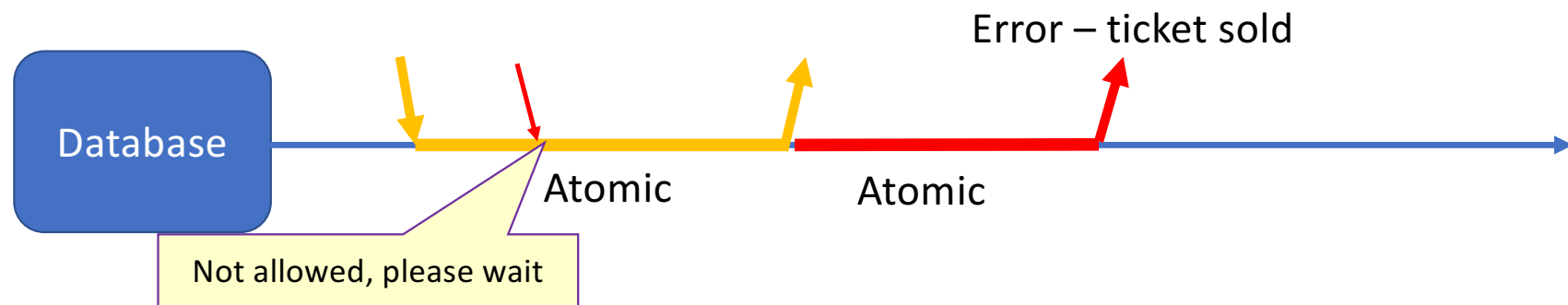
42



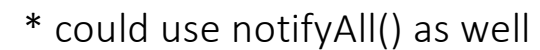
ATOMIC OPERATIONS

43

- Protecting your resources – one-at-a-time operation
- Once started, no-one shall access that specific resource until released



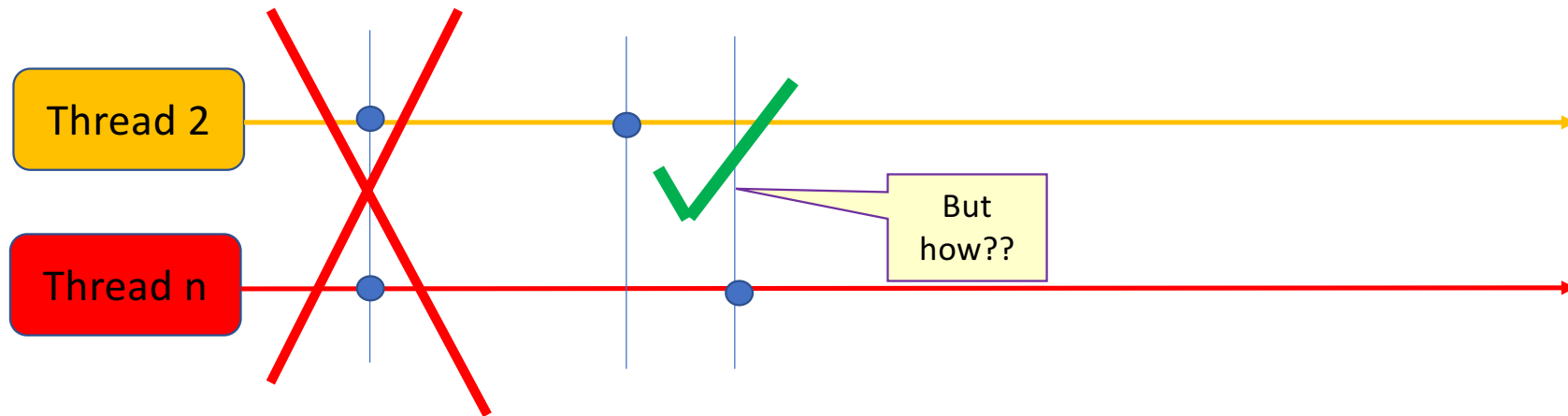
- Other examples (despite being so fast we do not notice) are
 - Network access (one packet at a time)
 - File I/O (one file read/write at a time)



NOTIFYALL() – NOT ENSURING FAIR HANDLING

45

- Threads 2 and 3 are in competition to get the last ticket



- Available options include
 - First to arrive has priority
 - Gold member(s) have priority
 - Randomness to who gets priority
 - Other??

AGENDA

46

- Multitasking
 - Processes and threads
 - Creating threads in Java
 - Thread life-cycles and methods
- Concurrency
 - Problems and solutions
 - Message sequence diagrams
 - Thread synchronization
- Examples
- The important stuff
- Assignments

THE IMPORTANT STUFF

- Threads run independently and unpredictably. The CPU scheduler interleaves execution in unpredictable ways, so never assume execution order
- Shared mutable state requires synchronization, if not then race conditions will occur (testability nightmare)
- Visibility is a real problem, so use synchronized, volatile, or atomic classes appropriately
- Prefer high-level concurrency utilities (Executors, Futures, Concurrent collections) that are readily provided in modern Java
- Poor synchronization causes deadlocks and bottlenecks — design concurrency intentionally and minimize shared state and hold locks

AGENDA

- Multitasking
 - Processes and threads
 - Creating threads in Java
 - Thread life-cycles and methods
- Concurrency
 - Problems and solutions
 - Message sequence diagrams
 - Thread synchronization
- Examples
- The important stuff
- Assignments



ASSIGNMENTS

Assignment 1

- a) Run the race-condition program found on Moodle
- b) Does it produce the desired output? Why not?
- c) Discuss ways to fix the program and make it count correctly
- d) Implement and test if your solution(s) works



ASSIGNMENTS

50

Assignment 2

- a) Design and implement user input for a train management program as a thread (you can call it a Command Line Input Handler) and the part which holds the list of trains in another thread. Make the program work with those two threads. Hint: create a class specific for 1) the user input and 2) the train list management.
- b) Create a set of threads of the user input handler to emulate a few train operators that wishes to seek, insert, delete trains from the main list. Make this run.
- c) Now automatize the user input to emulate 10-100's of train operators trying to query the train list (by either searching, inserting, deleting, updating bicycle entries). You should use the Random class to randomly select between the actions (previously taken as input from a real human via keyboard) a thread/train operator can take (if you forgot, then see previous assignment where you also used the Random class).
- d) How many threads/train operators can you handle before you start seeing problems? Is it possible to solve the issues that you observe? And to what level can your solution hold, i.e. how many threads can you handle before more issues shows up?